WEB APPLICATION SECURITY ASSESSMENT AND SQL INJECTION TESTING

Module 5 Assignment • DVWA Controlled Lab • Draft for Evidence Completion

Project	Web Application Security Assessment and SQL Injection Testing
Target	Damn Vulnerable Web Application (DVWA)
Lab address	http://172.20.10.5/dvwa/ (private lab network)
Security level	Low, as reported during manual testing
Status	Partial assessment — only observed results are claimed
Date	20 October 2026

Important: This draft documents results reported during the practical session. Attach your own screenshots before submission. Untested items are marked as pending and must not be presented as completed.

1. Executive Summary

A preliminary SQL injection assessment was conducted against DVWA in a controlled lab at http://172.20.10.5/dvwa/. During manual testing, a malformed quote input produced a visible MySQL syntax error. A Boolean expression returned multiple records, and a blind SQL injection page showed different visible responses for true and false conditions. These observations are consistent with SQL injection in the intentionally vulnerable DVWA training application.

Initial SQLMap runs did not confirm injection. The tool followed a redirect to the DVWA login page and did not use the authenticated browser session, making the automated result inconclusive for the intended vulnerable endpoint.

2. Scope and Methodology

Scope: the user's DVWA instance at the private lab address 172.20.10.5. Testing described here is limited to the intentionally vulnerable lab. No production system findings are claimed.

Tests reported: baseline input, malformed quote input, Boolean SQL injection input, true/false comparison in the blind module, GET parameter identification, and an initial unauthenticated SQLMap scan.

Limitations: raw HTTP captures and screenshots have not been embedded in this draft. A dedicated authentication-bypass test, POST parameter testing, and time-based blind SQL injection were not verified in the supplied results.

3. Results

Test	Input/action	Observed result	Interpretation
Baseline	SQLi: 1	ID 1; admin admin	Baseline response.
Malformed input	1'	MySQL syntax error near ''1''	Error disclosure; unsafe query construction indicated.
Boolean SQLi	1' OR '1'='1	Five records: admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith	Boolean-based SQLi behavior in DVWA Low.
Blind baseline	Blind SQLi: 1	admin admin	Baseline response.
Blind true	1' AND '1'='1	admin admin displayed	True condition returned record.
Blind false	1' AND '1'='2	No visible record	Response difference consistent with Boolean-based blind SQLi.
GET parameters	URL contains ?id=1&Submit=Submit	Parameters visible in URL	GET-style query parameters identified.
SQLMap initial	URL without authenticated session	302 redirect to login.php; no parameter confirmed injectable	Inconclusive for vulnerable endpoint.

4. Findings and Remediation

F-01 — SQL Injection behavior in DVWA SQL Injection module

Severity: Context-dependent. High impact is possible in a real application, but DVWA is intentionally vulnerable and is not a production target. **Evidence:** 1' OR '1'='1 returned five records while baseline 1 returned one; 1' also produced a MySQL syntax error. **Potential impact:** query logic manipulation and unauthorized record access depending on query context and database privileges. **Remediation:** parameterized queries/prepared statements, avoid SQL string concatenation, least-privilege database accounts, generic user-facing errors, protected server logs, and regression tests.

F-02 — Boolean-based blind SQL injection behavior

Evidence: 1' AND '1'='1 returned admin admin; 1' AND '1'='2 returned no visible record. **Potential impact:** an attacker may infer data from response differences. **Remediation:** parameterized queries, consistent response handling, least privilege, and automated tests.

F-03 — Database error disclosure

Evidence: malformed input displayed a MySQL syntax error. **Potential impact:** error details may reveal implementation information. **Remediation:** show generic errors to users and keep detailed exceptions in access-controlled logs.

5. SQLMap Automated Testing

The initial SQLMap command tested GET parameters id and Submit but received a 302 redirect to /dvwa/login.php. It then reported that neither parameter appeared injectable. Because the request did not include the authenticated browser session, this is not a conclusive negative result for the vulnerable page. The supplied follow-up command used -columns; the standard SQLMap option is --columns. No database enumeration success is claimed.

6. Remaining Assignment Items

• Dedicated authentication-bypass test and observed result.

• POST parameter testing if supported by the lab/module.

• Time-based blind SQL injection test and response-time comparison.

• Authenticated SQLMap test if required, after verifying the request reaches the intended endpoint.

• Attach genuine screenshots and ensure cookies/passwords are not visible.

• Prepare a short presentation or summary for the instructor.

7. Conclusion

Manual testing produced strong evidence of SQL injection behavior in the deliberately vulnerable DVWA Low-level lab. The initial SQLMap result was inconclusive due to the login redirect. Further evidence is needed to complete all assignment components. This report distinguishes observed findings from untested items.

8. Evidence Checklist

■ Screenshot 01 — baseline input 1 and admin admin result

■ Screenshot 02 — malformed input and MySQL syntax error

■ Screenshot 03 — Boolean SQLi input and five records

■ Screenshot 04 — blind true condition

■ Screenshot 05 — blind false condition

■ Screenshot 06 — URL showing GET parameters

■ Screenshot 07 — SQLMap initial scan and login redirect